

scheduled-work-engine – DESIGN

1

Revision: 1 Last modified: 2026-07-02T00:00:00Z

. Goal & scope

Backing infrastructure that gives the `REMINDER` / `BACKGROUND` action-prefix (§11.4.140) real teeth: a **decoupled, project-agnostic** (§11.4.28) service that tracks scheduled/background work items so an autonomous agent can, when a `REMINDER` fires, query for work whose outcome is **not yet confirmed** and re-verify it before reporting done (§11.4.6 no-guessing, §11.4.108 verify-before-report). No ATMOSphere/consumer-specific data is hardcoded; all configuration is via env/flags (§11.4.28).

. Architecture

<code>cmd/scheduled-work</code>	<code>subcommands: serve mcp-stdio acp-stdio </code>
<code>version</code>	
└─ <code>internal/server</code>	<code>gin.Engine (REST + POST /mcp) + brotli</code>
<code>middleware + TLS</code>	
└─ <code>internal/mcp</code>	<code>MCP JSON-RPC 2.0 dispatcher (shared by HTTP</code>
<code>+ stdio)</code>	
└─ <code>internal/acp</code>	<code>newline-delimited JSON-RPC stdio loop (MCP-</code>
<code>stdio + ACP-equiv)</code>	
└─ <code>internal/store</code>	<code>JSON-file store (mutex + atomic write), the</code>
<code>data model</code>	

One `gin.Engine` implements `http.Handler`, so it is served **identically** over HTTP/1.1, HTTP/2 (TCP TLS) and HTTP/3 (QUIC UDP). One MCP dispatcher backs the HTTP `/mcp` endpoint AND the stdio transports — a single tool implementation, three transports.

Current libraries (researched – § . . / § . . , verified – –)

- github.com/quic-go/quic-go **v0.60.0** —
`http3.Server{Handler, TLSConfig}` (`ListenAndServe` /
`Serve(net.PacketConn)`) for the server; `http3.Transport` (an
`http.RoundTripper`) for the client. gin does not serve HTTP/3 natively, so the
`gin.Engine` is wrapped as the `http.Handler` behind `http3.Server`
(confirmed against the vendored source, not assumed).
- github.com/andybalholm/brotli **v1.2.2** — `brotli.NewWriterLevel` ; note
it has no `WriteString` (the gin `ResponseWriter` `WriteString` is
implemented via `Write([]byte(s))`).
- github.com/gin-gonic/gin (latest) — router.
- github.com/google/uuid — item ids.

Graceful fallback: `serve` starts a TCP TLS `http.Server` (h1/h2) AND the UDP
`http3.Server` ; every h1/h2 response advertises the h3 alt-svc via
`SetQUICHeaders` , so clients upgrade to HTTP/3 when able. `SWQ_H3=0` runs h1/
h2 only.

11 4 6

. Store choice – JSON file (justification, § . .)

A reminder / scheduled-work queue is a **low-write-volume** workload. Options:

- **SQLite (mattn/go-sqlite3)** — needs CGO, complicating static/cross builds.
- **BoltDB** — an extra module dependency for a tiny dataset.
- **JSON file (chosen)** — zero CGO, zero extra module, trivially portable across
every supported platform (§11.4.81), human-inspectable for debugging.

Durability: a `sync.RWMutex` serialises all access; writes go through write-
temp- `fsync` -rename (atomic replace), so a crash never leaves a torn file. `Open`
on a corrupt file returns an error rather than silently dropping data (§11.4.6). The
`TestConcurrentCreate` test drives 50 concurrent creates through the mutex with
zero lost writes; `go test -race` is clean.

. ACP surface – honesty note

(§ . . / § . .)

"ACP" (Agent Client Protocol, agentclientprotocol.com) is an emerging JSON-RPC-2.0-over-stdio protocol whose full session / prompt-turn model targets interactive coding **agents**, not a stateless work-queue service. Implementing the complete ACP agent lifecycle here would invent behaviour that does not fit this service. Instead `internal/acp` exposes the **same** newline-delimited JSON-RPC-2.0-over-stdio transport ACP uses, dispatching to the identical tool set as the MCP surface. It is therefore an **ACP-EQUIVALENT** adapter, clearly labelled in source — not a claim of full ACP-agent conformance. This is stated as a fact, not hidden behind a green test.

. Anti-bluff test coverage

(§ . / § . . / § .)

- **Unit** (`internal/store` , `internal/mcp`) — model, lifecycle, derived `overdue` , needs-verification, persistence round-trip, invalid-input rejection, concurrency; MCP initialize / tools-list / tools-call / error paths.
- **Integration** (`test/` , no mocks — §11.4.27) — boots the REAL `http3.Server` on an ephemeral UDP port and round-trips `create → list → mark-done → verify` over **HTTP/3** (asserts `resp.Proto == "HTTP/3.0"`) with **brtli** (asserts `Content-Encoding: br` and decodes it); plus the MCP-over-HTTP/3 endpoint.
- **§1.1 paired mutation** (`test/meta_mutation.sh`) — breaks the mark-done handler at source, asserts the suite goes RED, restores, asserts GREEN. Proves the suite is not a bluff gate.
- **Live smoke** (`test/live_smoke.sh`) — boots the built binary and confirms the full lifecycle over REST-TLS + MCP-stdio, capturing the transcript under `qa-results/<ts>/` .

Honest boundary: brotli-over-HTTP/3 is asserted programmatically in the Go integration test (a real h3 client); the `curl` live smoke uses the h1/h2 fallback of the same engine because curl does not speak HTTP/3 to a self-signed cert here.

. Consumer wiring

- MCP registration: `constitution/mcp/scheduled-work-mcp.json` .
- Claude Code plugin: `constitution/plugins/scheduled-work/` (`.claude-plugin/plugin.json` + `.mcp.json` + `build.sh`).
- Skill: `constitution/skills/scheduled-work-queue/` (`skill.md` + `register.sh`).

. Proposed governance-anchor handoff (for the conductor – NOT applied here)

Per the task's file-scope isolation, this component does NOT edit Constitution.md or any carrier. If the operator/conductor wishes to make this infrastructure a mandated capability, the following anchor text is PROPOSED for integration by the conductor (classification: universal, §11.4.17):

§11.4.NNN — Scheduled-work / reminder verify-before-report backing service. The `REMINDER` / `BACKGROUND` action-prefix (§11.4.140) MUST be backed by a durable scheduled-work queue (the `scheduled-work-engine`) so that deferred / background work is recorded with a lifecycle status, and a `REMINDER` MUST query `list_needs_verification` (`blocked` / `uncertain` / `overdue`) and re-confirm each item's real outcome with captured evidence (§11.4.5/§11.4.69) BEFORE the agent reports it done (§11.4.6/§11.4.108) — never marking an item done on assumption. The queue is decoupled + project-agnostic (§11.4.28), consumed by reference, and the loop done-condition (§11.4.87/§11.4.126) MUST NOT read satisfied while any tracked item is non-terminal. Recommended gate `CM-SCHEDULED-WORK-VERIFY-BEFORE-DONE` + paired §1.1 mutation. Composes §11.4.6/§11.4.28/§11.4.108/§11.4.116/§11.4.126/§11.4.140.

. Cross-references

§11.4.6 · §11.4.8 · §11.4.28 · §11.4.30 · §11.4.77 · §11.4.81 · §11.4.108 · §11.4.116
· §11.4.126 · §11.4.140 · §11.4.164 · §1.1.